Experiment No : 10

Title : Implement UART communication using Data Structures in C

Name of Student: Shamiksha Chaudhari
Class: SY **Div:** SY1 **PRN No:** B25ET1005 **Batch:** A1

Program:

```
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

#define UART_BUFFER_SIZE 64   // Size of TX and RX buffers

// -------------------------
// Circular Queue Structure
// -------------------------
typedef struct {
    uint8_t buffer[UART_BUFFER_SIZE];
    int head;
    int tail;
    int count;
} UART_Buffer;

// -------------------------
// UART Simulation Structure
// -------------------------
typedef struct {
    UART_Buffer txBuffer;   // Transmit buffer
    UART_Buffer rxBuffer;   // Receive buffer
} UART_Device;

// -------------------------
// Queue Functions
// -------------------------
void UART_Buffer_Init(UART_Buffer *buf) {
    buf->head = 0;
    buf->tail = 0;
    buf->count = 0;
}

bool UART_Buffer_IsFull(UART_Buffer *buf) {
```

```
    return buf->count == UART_BUFFER_SIZE;
}

bool UART_Buffer_IsEmpty(UART_Buffer *buf) {
    return buf->count == 0;
}

bool UART_Buffer_Enqueue(UART_Buffer *buf, uint8_t data) {
    if (UART_Buffer_IsFull(buf)) return false; // buffer full
    buf->buffer[buf->head] = data;
    buf->head = (buf->head + 1) % UART_BUFFER_SIZE;
    buf->count++;
    return true;
}

bool UART_Buffer_Dequeue(UART_Buffer *buf, uint8_t *data) {
    if (UART_Buffer_IsEmpty(buf)) return false; // buffer empty
    *data = buf->buffer[buf->tail];
    buf->tail = (buf->tail + 1) % UART_BUFFER_SIZE;
    buf->count--;
    return true;
}

// -------------------------
// UART Functions
// -------------------------
void UART_Init(UART_Device *uart) {
    UART_Buffer_Init(&uart->txBuffer);
    UART_Buffer_Init(&uart->rxBuffer);
    printf("UART Initialized.\n");
}

// Simulate sending a byte over UART
void UART_SendByte(UART_Device *uart, uint8_t data) {
    if (UART_Buffer_Enqueue(&uart->txBuffer, data))
        printf("TX: '%c' enqueued\n", data);
    else
        printf("TX Buffer Full! Data lost.\n");
}

// Simulate receiving a byte from UART hardware
void UART_ReceiveByte(UART_Device *uart, uint8_t data) {
    if (UART_Buffer_Enqueue(&uart->rxBuffer, data))
        printf("RX: '%c' received\n", data);
```

```
    else
        printf("RX Buffer Full! Data lost.\n");
}

// Simulate UART transmitting data (sending from TX buffer)
void UART_ProcessTransmit(UART_Device *uart) {
    uint8_t data;
    while (UART_Buffer_Dequeue(&uart->txBuffer, &data)) {
        printf("Transmitted: '%c'\n", data);
    }
}

// Simulate UART reading data (from RX buffer)
void UART_ReadReceivedData(UART_Device *uart) {
    uint8_t data;
    while (UART_Buffer_Dequeue(&uart->rxBuffer, &data)) {
        printf("Read from RX: '%c'\n", data);
    }
}

// -------------------------
// Main Function
// -------------------------
int main() {
    UART_Device uart;
    UART_Init(&uart);

    // Simulate sending data
    UART_SendByte(&uart, 'H');
    UART_SendByte(&uart, 'E');
    UART_SendByte(&uart, 'L');
    UART_SendByte(&uart, 'L');
    UART_SendByte(&uart, 'O');

    // Process TX (simulate actual transmission)
    UART_ProcessTransmit(&uart);

    // Simulate receiving data
    UART_ReceiveByte(&uart, 'O');
    UART_ReceiveByte(&uart, 'K');

    // Process RX (read incoming data)
    UART_ReadReceivedData(&uart);
```

```
    return 0;
}
```

OUTPUT:

```
UART Initialized.
TX: 'H' enqueued
TX: 'E' enqueued
TX: 'L' enqueued
TX: 'L' enqueued
TX: 'O' enqueued
Transmitted: 'H'
Transmitted: 'E'
Transmitted: 'L'
Transmitted: 'L'
Transmitted: 'O'
RX: 'O' received
RX: 'K' received
Read from RX: 'O'
Read from RX: 'K'
```